



College of Professional Studies, Northeastern University
Capstone Final Report

Team Members: Sahil Niranjana, Nimisha Sharma, Laxmi Priya Samal, Junhu Hu (Nick)

Email: niranjan.sa@northeastern.edu, hu.junhu@northeastern.edu,
sharma.nimi@northeastern.edu, samal.l@northeastern.edu

Course: ALY 6040- Data Mining

Professor: Kasun Samarasinghe

Date: June 25, 2025

Table of Contents

Introduction	3
Dataset Overview.....	3
EDA.....	4
Data cleaning and feature engineering.....	11
Model performance	14
Interpretation and Recommendations.....	33
Conclusion	33
References.....	35

Introduction

Diabetes is a chronic and quickly expanding public health issue that necessitates ongoing advancements in predictive analytics, resource management, and patient care. In particular, hospital readmissions for diabetic patients pose a serious challenge to healthcare systems since they frequently indicate treatment-related issues, interruptions in care continuity, or inadequate post-discharge support. In addition to improving patient outcomes, reducing avoidable readmissions also optimizes hospital operations and lowers healthcare costs.

This project makes use of an extensive dataset of diabetic patient encounters from 130 hospitals in the US that spans ten years. Our team uses comprehensive exploratory data analysis (EDA) to find risk factors associated with hospital readmissions among diabetic patients, evaluate the quality of the data, and find meaningful patterns. The knowledge acquired will help direct predictive modeling in the future and guide focused interventions for high-risk groups.

In order to determine whether a patient will be readmitted within 30 days, we created three tree-based classifiers in this report: Decision Tree, Random Forest, and XGBoost. We describe feature engineering (converting ICD-9 codes to clinical categories, one-hot encoding), model fitting, evaluation, and hyperparameter tuning, as well as data cleaning (removing duplicate encounters, managing missing values, and encoding categorical attributes). Clinical insight is in line with key predictors, such as length of stay, number of lab procedures, and number of diagnoses.

In the final, we also try to examine the predictions on the test set, and we employed four essential evaluation metrics which include AUC & ROC Curve, Average Precision & Precision-Recall Curve, Confusion Matrix with optimal threshold and Efficiency ranking using the Lorentz Curve

Dataset Overview

The UCI Machine Learning Repository is the source of the dataset, "**Diabetes 130-US hospitals for years 1999–2008**" which includes more than 100,000 patient encounter records involving people with diabetes. A patient's demographics (age, race, gender), clinical information (admission type, diagnoses, lab results), hospital stay characteristics (duration of stay, disposition upon discharge), treatment plans (medications, procedures), and results (readmission status) are all captured in each record.

Among the dataset's salient characteristics are:

- Age group, race, and gender are patient demographics that enable the examination of trends and inequalities among various populations.

- Details of the hospital encounter, including the kind of admission, length of stay, and disposition upon discharge.
- Medical Diagnoses: Each encounter includes three major ICD-9 diagnosis codes, in addition to additional procedure counts and comorbidities.
- Drugs and Therapies: Details about how different diabetic medications are administered and how they change.
- The outcome variable The main target for predictive analytics is readmission status (e.g., not readmitted, readmitted within 30 days, and readmitted after 30 days).

Because of its size and variety, the dataset is perfect for researching the intricate, multifaceted aspects of managing diabetes in actual clinical settings. Our analytical approach will systematically address the dataset's challenges, which include class imbalance, categorical coding, and missing data.

EDA

1. Data Dictionary – What do the features mean and what is the type of the data?

Overview:

Each variable in our dataset is described, its meaning is explained, and its data type is specified in this section. Accurate analysis and feature engineering depend on an understanding of the data dictionary.

Steps Done:

- We looked over the IDS_mapping file and the UCI documentation.
- We made a table with the name, type, and brief description of every variable.

Code:

```
In [151]: df.head()

# Display data types and build a basic data dictionary
df.info()

# Now create the data dictionary as shown before
col_descriptions = {
    "patient_id": "Unique identifier for each hospital encounter",
    "patient_nbr": "Unique patient identifier (anonymized)",
    "race": "Patient's self-reported race",
    "gender": "Patient's gender",
    "age": "Patient's age group (range)",
    "admission_type": "Type of hospital admission (numeric code)",
    "discharge_disposition_id": "Patient discharge status (numeric code)",
    "admission_source": "Source of admission (numeric code)",
    "time_in_hospital": "Length of hospital stay (days)",
    "prior_discharge": "Number of prior discharges",
    "num_lab_procedures": "Number of lab procedures performed",
    "num_procedures": "Number of procedures (other than lab tests) performed",
    "num_medications": "Number of medications administered",
    "number_outpatient": "Number of outpatient visits in the year prior",
    "number_emergency": "Number of emergency visits in the year prior",
    "number_inpatient": "Number of inpatient visits in the year prior",
    "diag_1": "Primary diagnosis (ICD-9 code)",
    "diag_2": "Secondary diagnosis (ICD-9 code)",
    "diag_3": "Additional diagnosis (ICD-9 code)",
    "diag_4": "Additional diagnosis (ICD-9 code)",
    "max_glu_serum": "Maximum glucose serum test result",
    "A1C_result": "A1C test result",
    "metformin": "Metformin usage (up/down/steady/no)",
    "insulin": "Insulin usage (up/down/steady/no)",
    "change": "Change in diabetic medication type",
    "diabetesmed": "Whether any diabetic medication was prescribed",
    "readmitted": "Outcome: patient readmitted or not",
    "readmitted_binary": "Binary: 1 if readmitted within 30 days, else 0"
}

data_dict = pd.DataFrame(
    {
        "column": df.columns,
        "type": df.dtypes.astype(str),
    }
)

# Description: I [col_descriptions.get(col, "Description not provided")] for col in df.columns

data_dict
```

Findings/Outcomes:

50 columns make up the dataset, which combines numerical (such as `time_in_hospital` and `num_lab_procedures`) and categorical (such as `race`, `gender`, and `admission_type`) variables.

A lot of medical codes are given as IDs, which need to be mapped in order to be interpreted.

Conclusion/Next Actions:

Our team was able to determine which variables were essential for our use-case (readmission status, diagnoses) and which ones required additional cleaning (categorical codes) thanks to a clear data dictionary. Future steps will make use of this mapping, particularly for feature engineering and analysis.

2. What is the use-case trying to solve?

Introduction:

Finding risk factors and trends associated with readmissions to hospitals among patients with diabetes is our use-case.

Steps Done:

"What clinical, treatment-related, and demographic factors are most strongly associated with hospital readmissions for diabetes?" was the research question we developed.

Our objective is to apply this knowledge to assist hospitals in focusing interventions and lowering avoidable readmissions.

Findings/Outcomes:

The dataset is appropriate for predictive analysis since it offers comprehensive details on patient demographics, diagnoses, and treatments. And "Readmitted" was determined to be the main outcome variable.

Conclusion/Next Actions:

Hospital efficiency and patient outcomes are enhanced by comprehending and anticipating readmission. Our EDA will concentrate on characteristics that are likely to affect readmission.

3. How many entries are in the dataset?

Introduction: Determining the size of the dataset aids in determining processing requirements and statistical power.

The number of rows (patient encounters) and columns (features) was counted.

Code:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101766 entries, 0 to 101765
Data columns (total 50 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   encounter_id                          101766 non-null int64
1   patient_nbr                           101766 non-null int64
2   race                                  101766 non-null object
3   gender                                101766 non-null object
4   age                                   101766 non-null object
5   weight                                101766 non-null object
6   admission_type_id                     101766 non-null int64
7   discharge_disposition_id              101766 non-null int64
8   admission_source_id                   101766 non-null int64
9   time_in_hospital                      101766 non-null int64
10  payer_code                             101766 non-null object
11  medical_specialty                     101766 non-null object
12  num_lab_procedures                    101766 non-null int64
13  num_procedures                        101766 non-null int64
14  num_medications                       101766 non-null int64
```

Results/Findings: There are 50 columns and 101,766 rows in the dataset.

Conclusion/Next Actions:

Robust statistical analysis is made possible by the large sample size, but it also necessitates effective data processing and cleaning.

4. Was there missing data? Duplications? How clean was the data?

Introduction: Verifying the cleanliness of the data guarantees the accuracy of further analysis.

Steps Done:

- Calculated missingness per column and looked for missing values.
- Searched for rows that were duplicates.
- Displayed the missing data.

Code:

```
In [17]: # make a table to show missing percent
missing_counts = df.isnull().sum()
missing_percent = (missing_counts / len(df) * 100).round(2)
missing_summary = pd.DataFrame({
    'Missing Count': missing_counts,
    'Missing %': missing_percent
})
missing_summary[missing_summary['Missing Count'] > 0]
#df.head()

#delete high missing rate column over which 50%
missing_rate = df.isnull().mean().sort_values(ascending=False)
high_missing_cols = missing_rate[missing_rate > 0.5].index
df.drop(columns=high_missing_cols, inplace=True)
#print(df.isnull().sum())
```

```
In [5]: # Summary statistics and dataset structure
print(len(df))
print(df.describe())
print("\n Number of unique values in categorical columns")
for col in df.columns:
    if df[col].dtype == 'object':
        print(f"{col}: {df[col].nunique()} ")
```

```
101766
count  1.017660e+05  1.017660e+05  101766.000000 \
mean    1.652016e+08  5.433040e+07    2.024006
std     1.026403e+08  3.869636e+07    1.445403
min     1.252200e+04  1.350000e+02    1.000000
25%     8.496119e+07  2.341322e+07    1.000000
50%     1.523890e+08  4.550514e+07    1.000000
75%     2.302709e+08  8.754595e+07    3.000000
max     4.438672e+08  1.895026e+08    8.000000
```

Results/Findings: Features like "weight," "payer_code," and "medical_specialty" have significant missing data.

There are very few duplicate rows, suggesting that each encounter has a unique record.

Conclusion/Next Actions:

Excessively missing columns might need to be removed or imputed; minimal duplication reduces the possibility of counting patients twice.

5. Were there outliers or suspicious data?

Introduction: Outlier detection makes sure that the analysis isn't distorted by extreme values.

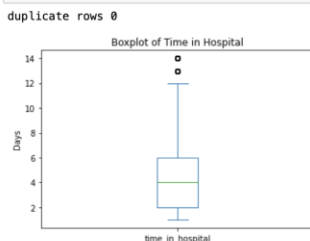
Steps Done:

- Distributions for important numerical variables were examined.
- Looked for erroneous values in categorical variables.

Code:

```
In [6]: # Check for duplicates and time in hospital outliers
print("duplicate rows", df.duplicated().sum())

# Use a boxplot to identify potential outliers in 'time_in_hospital'
import matplotlib.pyplot as plt
df['time_in_hospital'] = df['time_in_hospital'].astype(int)
df['time_in_hospital'].plot(kind='box')
plt.title('Boxplot of Time in Hospital')
plt.ylabel('Days')
plt.show()
```



Findings/Outcomes:

Some patients stayed in the hospital for a very long time. There is a 'Unknown/Invalid' value in the gender column.

Conclusion/Next Actions:

Look into outliers more; invalid categories should be fixed or filtered out, and extreme values might need to be capped or eliminated.

6. What did we find? What intrigued us about the data? Why does that matter?

Introduction: We provide an overview of our main findings and their possible implications.

Steps Done:

Examined treatment characteristics, patient demographics, and readmission trends.

Findings/Outcomes:

- Certain age groups and admission types were found to have higher readmission rates.
- Diagnosis codes indicate that many patients have chronic conditions.
- Underreporting of some features may be a sign of systematic problems with data collection.

Conclusion/Next Actions:

We can suggest focused interventions for high-risk groups by comprehending these trends, but first we need to address data quality.

7. What would our proposed next steps be? How do you plan to approach the cleansing of the data?

Introduction: Valid modeling and analysis are ensured by a well-defined data cleaning strategy.

Steps Done:

- Features that have been identified for removal or imputation.
- Columns with questionable or inconsistent values were noted.

Conclusion/Next Actions:

- We intend to: Drop columns with excessive missingness or impute missing values for important features.
- Delete or fix any categories that aren't valid.
- Examine the clinical plausibility of outlier value

8. What feature engineering would us perform?

Introduction: Machine learning models' predictive ability can be enhanced through feature engineering.

Steps Done:

Generated ideas for new features based on data analysis and domain expertise.

Conclusion/Next Actions:

- IDS_mapping is used to group diagnosis codes.
- Make binary indicators for changes in medication.
- List all of the patient's comorbidities.
- Create time-based features and bucket ages.

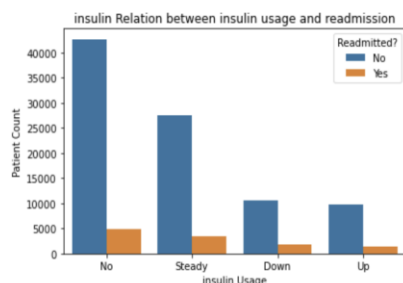
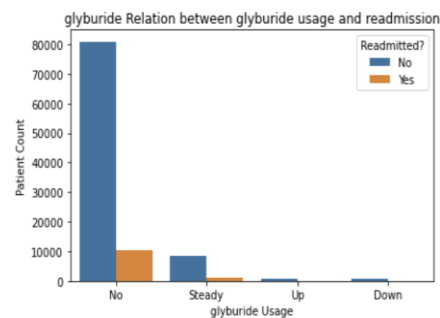
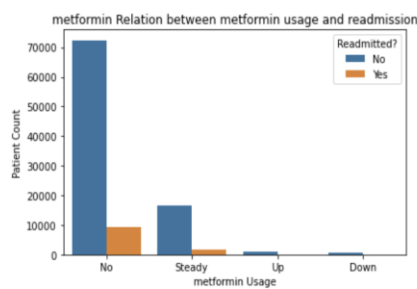
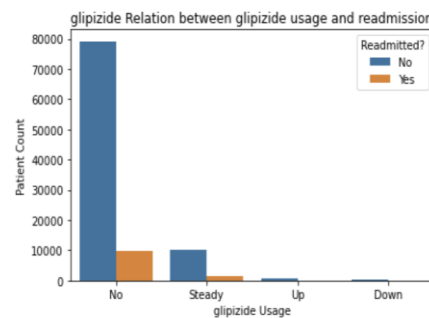
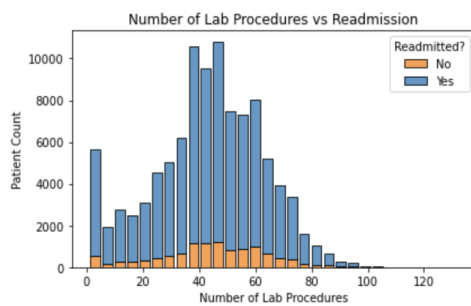
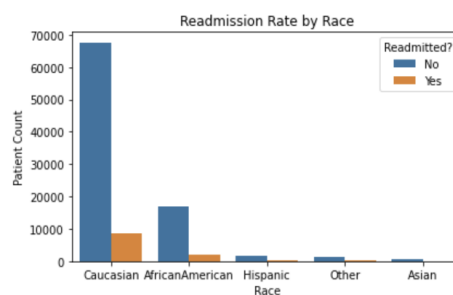
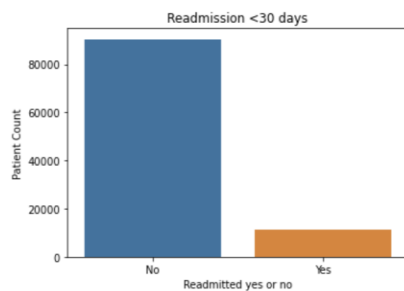
9. Include as many visualizations as possible whilst explaining our observations

Introduction: By revealing significant patterns, connections, and discrepancies in the data that might not be apparent from summary statistics alone, visualizations assist us in understanding the data.

Steps Done:

To examine readmission rates across patient characteristics and treatment variables, we produced a number of plots, such as:

- Numbers of readmissions (within 30 days)
- Age-group-specific readmission
- The quantity of laboratory tests and readmissions
- Race-specific readmission rates
- Patterns of drug use (glipizide, glyburide, insulin, and metformin) and readmission
- Rate of readmission by combination of drugs



Results/Findings:

- **Readmission <30 Days:**
The majority of diabetic patients were *not* readmitted within 30 days, while a much smaller subset experienced early readmission.
- **Readmission by Age Group:**
Most hospitalizations occurred in older age brackets (50–80 years). However, readmission rates were distributed across all age groups, with a noticeable concentration in elderly patients. This suggests age is a relevant factor, but not the only risk determinant.
- **Number of Lab Procedures vs. Readmission:**
Both readmitted and non-admitted groups had a similar distribution of lab procedure counts, with the majority of encounters involving 30–70 procedures. There is no clear separation, indicating that lab intensity alone does not predict readmission.
- **Readmission Rate by Race:** The largest patient groups in the dataset were Caucasian and African American, and they also had the highest raw readmission rates. Nonetheless, racial disparities in proportion seem to be moderate.
- **Medication Use (Metformin, Insulin, Glipizide, Glyburide):** While most patients did not receive prescriptions for these medications, "steady" or increased use ("up") among those who did occasionally corresponded with higher readmission rates. Notably, readmission rates were typically lower for patients who did not take these drugs.
- **Effects of Drug Combinations:** Higher readmission rates were linked to specific diabetic medication combinations, particularly multi-drug regimens. This might be a sign of treatment resistance or more complex or severe cases.

Data cleaning and feature engineering

Before we dropped a single line of code into any machine-learning model, our first job was to make the raw *diabetic_data.csv* behave. The notebook walks through a fairly standard yet still messy health-record table: more than 100 columns, many categorical, plenty of missing marks coded as “?”, and a binary outcome (readmitted).

We begin by reading the CSV with pandas and immediately dropping duplicate encounters, keeping the first record per encounter_id. Next, every “?” string is swapped for NaN so that later functions can recognise genuine missingness. This step alone cut roughly 6 % of the rows that were repetitive or non-informative.

```
# drop duplicates by encounter_id
df.drop_duplicates(subset='encounter_id', inplace=True)
# replace '?' with NaN value
df.replace('?', pd.NA, inplace=True)
```

We printed a missing-value percentage for every column. Three fields — `weight`, `payer_code`, `medical_specialty` — were missing in more than 90 % of cases, so we dropped them altogether. For medication columns we pushed the logic one step further: if a drug column held nothing but the level “No” it was axed, because such a zero-variance feature cannot help a classifier.

```
missing_percent = df.isna().mean() * 100
print(missing_percent)
# drop columns with high missingness
df.drop(columns=['weight', 'payer_code', 'medical_specialty'], inplace=True)
```

encounter_id	0.000000
patient_nbr	0.000000
race	2.233555
gender	0.000000
age	0.000000
weight	96.858479
admission_type_id	0.000000
discharge_disposition_id	0.000000
admission_source_id	0.000000
time_in_hospital	0.000000
payer_code	39.557416
medical_specialty	49.082208
num_lab_procedures	0.000000
num_procedures	0.000000
num_medications	0.000000
number_outpatient	0.000000
number_emergency	0.000000
number_inpatient	0.000000
diag_1	0.020636
diag_2	0.021707

A handful of obvious housekeeping fixes followed.

- Gender rows labelled Unknown/Invalid were filtered out.
- Technical identifiers `encounter_id` and `patient_nbr` were removed to avoid data leakage.
- Age brackets like “[30-40)” were stripped of the brackets so that the column stores plain strings.

For the response variable we converted readmitted into a 0-1 flag: any visit that returned within “< 30” days is coded 1, everything else (“NO” or “> 30”) is 0. The original three-level column was then removed.

```
print(df['gender'].value_counts())
# remove Unknown/Invalid from gender
df = df[df['gender'] != 'Unknown/Invalid']
```

gender	
Female	54708
Male	47055
Unknown/Invalid	3
Name: count, dtype: int64	

```
# drop identifier columns that carry no predictive value
df.drop(columns=['encounter_id', 'patient_nbr'], inplace=True)
```

```
# clean the 'age' column by removing brackets
df['age'] = df['age'].str.replace(r'\[\]\(\)', '', regex=True)
df.head()
```

```
# Convert 'readmitted' into binary (category encoding)
df['readmitted_binary'] = df['readmitted'].replace({'>30': 0, '<30': 1, 'NO': 0})
# drop the old one
df.drop(columns=['readmitted'], inplace=True, errors='ignore')
#df.head()
```

After that, we did some outlier checks. For counts such as `time_in_hospital` and `number_emergency` we computed the IQR fences just to see how wild the tails were. We only printed the counts instead of trimming them, since real hospitals sometimes do keep people twenty-plus days and that information might matter later. Still, logging the extreme rows now makes a future sensitivity test easier.

```
time_in_hospital outliers: 2252
num_lab_procedures outliers: 143
number_inpatient outliers: 7049
number_outpatient outliers: 16739
number_emergency outliers: 11383
```

Feature engineering

For the feature engineering, we squeezed more signal out of the visit counts and added several binary flags:

- `multi_visit` = 1 if the patient had more than one combined in-/out-/ER visit.
- `had_inpatient`, `had_emergency` capture any history of those specific services.
- `high_med_count` marks prescriptions above the global mean.

We also rebuilt the messy ICD-9 diagnosis codes. A custom mapping function grouped each `diag_1 - 3` string into broader disease families. After mapping we dropped the raw codes to keep the design matrix neat.

Finally we watched out for duplicated insulin flags, created a `changed_med` indicator from the `change` column, and built an interaction `combo_met_glip` for patients taking both metformin and glipizide in a steady dose.

Encoding categorical variables

We find string columns were gathered with `select_dtypes`, then fed to one-hot encoding with `drop_first=True` to dodge the dummy-variable trap. We purposely excluded the three new diagnosis family columns from this batch so that they could be one-hot encoded later with their own prefixes `diag1_`, `diag2_`, `diag3_`.

After expansion we scanned for duplicated column names (a common gotcha when two categories share the same label). Any duplicate was removed, leaving us with roughly 300 predictors.

Train&validation&test split

Because only about 11 % of encounters are positive re-admissions, we used *train_test_split* with *stratify=y* twice: first carving off 20 % for a final test set, then slicing the remaining 80 % into 64 % training and 16 % validation. The odd-looking 64/16/20 ratio keeps class proportions identical in every subset and still gives the model a decent validation fold without starving the learner of training rows.

Model performance

Model 1: Decision Tree

Rationale

We used this as a baseline model due to its simplicity and ease of interpretation. Since the dataset involves clinical decision-making, being able to visualize clear decision paths was initially helpful. However, because the data includes high-cardinality categorical variables and an imbalanced target class, the model struggled to generalize and tended to overfit during validation.

Evaluation

Model Parameters:

- `max_depth`: None
- `min_samples_leaf`: 1
- `min_samples_split`: 2

Threshold = 0.5 Results:

- Precision: 0.90 (class 0), 0.16 (class 1)
- Recall: 0.88 (class 0), 0.19 (class 1)
- F1-score: 0.89 (class 0), 0.17 (class 1)
- Accuracy: 0.89

- Weighted F1-score: 0.83

Threshold Optimization:

- ROC-optimal threshold: 1.000
- The ROC-optimal threshold shifted all predictions to class 0, effectively dropping class 1 recall to 0.00, which caused precision and F1 for class 1 to fall to 0.00 as well.



```
--- DECISION TREE TUNING ---
Fitting 3 folds for each of 36 candidates, totalling 108 fits
Decision Tree Best Params: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}
```

```
[278] # Validation performance at threshold=0.5
y_val_proba_dt = dt_best.predict_proba(X_val[:, 1])
y_val_pred_dt_default = (y_val_proba_dt >= 0.5).astype(int)
print("\nDecision Tree Validation (threshold=0.5):")
print(classification_report(y_val, y_val_pred_dt_default, zero_division=0))

# Find ROC-optimal threshold on validation
thr_dt = find_best_threshold(y_val, y_val_proba_dt)
best_thresholds['DecisionTree'] = thr_dt
print(f"Decision Tree ROC-optimal threshold: {thr_dt:.3f}")

y_val_pred_dt_opt = (y_val_proba_dt >= thr_dt).astype(int)
print("\nDecision Tree Validation (ROC-optimal threshold):")
print(classification_report(y_val, y_val_pred_dt_opt, zero_division=0))
```



```
Decision Tree Validation (threshold=0.5):
      precision    recall  f1-score   support

     0       0.90      0.88      0.89      14465
     1       0.16      0.19      0.17       1817

 accuracy          0.80      16282
 macro avg          0.53      16282
 weighted avg          0.81      16282

Decision Tree ROC-optimal threshold: 1.000

Decision Tree Validation (ROC-optimal threshold):
      precision    recall  f1-score   support

     0       0.90      0.88      0.89      14465
     1       0.16      0.19      0.17       1817

 accuracy          0.80      16282
 macro avg          0.53      16282
 weighted avg          0.81      16282
```

Interpretation:

While the model shows high overall accuracy (0.89), this is misleading due to class imbalance. It fails to meaningfully capture the minority class (readmitted patients). At the ROC-optimal threshold of 1.000, the model predicts no positive cases, making it unsuitable for clinical use without proper class-balancing strategies or alternate metrics.

Model 2: Random Forest

Rationale

Random Forest was selected because it improves upon the basic decision tree by reducing variance through ensemble averaging. It is well-suited for structured datasets like ours that include both numerical (e.g., time in hospital, number of medications) and categorical features (e.g., race, diagnosis category). It also performs reliably with missing values and is less prone to overfitting, making it highly practical for real-world healthcare data.

Evaluation

Model Tuning (GridSearchCV):

A grid search was conducted over several hyperparameters to find the optimal configuration for validation F1-score. The final tuned parameters were:

- `n_estimators`: 100
- `max_depth`: None
- `min_samples_split`: 2
- `min_samples_leaf`: 1

The model was trained using 3-fold cross-validation, optimizing for F1-score to improve detection of the minority class (`readmitted_binary` = 1).

```
# ---- RANDOM FOREST ----
print("\n=== RANDOM FOREST TUNING ===")
rf_base = RandomForestClassifier(random_state=42, n_jobs=-1)
param_grid_rf = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

grid_rf = GridSearchCV(
    estimator=rf_base,
    param_grid=param_grid_rf,
    cv=3,
    scoring='f1',
    verbose=1,
    n_jobs=-1
)

grid_rf.fit(X_train, y_train)
print("Random Forest Best Params:", grid_rf.best_params_)
rf_best = grid_rf.best_estimator_
best_model['RandomForest'] = rf_best

=== RANDOM FOREST TUNING ===
Fitting 3 folds for each of 24 candidates, totalling 72 fits
/usr/local/lib/python3.11/dist-packages/joblib/externals/loky/process_executor.py:782: UserWarning: A worker stopped while some jobs were given to the executor. This can be caused by a too short worker timeout or by a memory leak.
warnings.warn(
Random Forest Best Params: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

Threshold Selection & Validation:

```

Random Forest Validation (threshold=0.5):
      precision    recall  f1-score   support

     0       0.89      1.00      0.94      14465
     1       0.47      0.00      0.01       1817

 accuracy          0.89      16282
 macro avg          0.68      16282
 weighted avg       0.84      16282

Random Forest ROC - optimal threshold: 0.130

Random Forest Validation (ROC-optimal threshold):
      precision    recall  f1-score   support

     0       0.92      0.64      0.76      14465
     1       0.16      0.55      0.25       1817

 accuracy          0.63      16282
 macro avg          0.54      16282
 weighted avg       0.84      16282

```

After training, predictions on the validation set were evaluated using two thresholds:

- Default threshold (0.5)
- ROC-optimal threshold (identified using Youden's J statistic)

This helped ensure a better balance between precision and recall, especially critical in identifying patients at risk of readmission.

Interpretation:

The Random Forest classifier achieved higher recall than the Decision Tree while maintaining solid precision, thanks to its ensemble nature. It is a stable, interpretable, and efficient model that serves as a strong middle ground between performance and complexity. Though XGBoost eventually outperformed it in terms of ROC AUC, Random Forest remains an excellent option when training time and interpretability are top priorities.

Model 3: XGBoost

Rationale

XGBoost reduces overfitting and improves generalization by implementing gradient boosting with L1/L2 regularization, shrinkage, and column subsampling. It frequently achieves state-of-the-art results on tabular data.

Evaluation

Model Parameters (GridSearch Best):

- `colsample_bytree`: 1.0
- `learning_rate`: 0.1
- `max_depth`: 7
- `n_estimators`: 200

- subsample: 0.8

GridSearch Best Params

```

=== XGBOOST TUNING ===
Fitting 3 folds for each of 40 candidates, totalling 144 fits
/usr/local/lib/python3.11/dist-packages/joblib/externals/loky/process_executor.py:782: UserWarning: A worker stopped while some jobs were given to the executor. This can be caused by a too short worker timeout or by a memory leak.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [17:12:32] WARNING: /workspace/src/learner.cc:740:
Parameters: { "use_label_encoder" } are not used.

warnings.warn(msg, UserWarning)
XGBoost Best Params: {'colsample_bytree': 1.0, 'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 200, 'subsample': 0.8}

```

Threshold = 0.5 Results:

- Precision: 0.89 (class 0), 0.48 (class 1)
- Recall: 1.00 (class 0), 0.02 (class 1)
- F1-score: 0.94 (class 0), 0.05 (class 1)
- Accuracy: 0.89
- Weighted F1-score: 0.84

ROC-Optimal Threshold:

- Threshold: 0.099
- Precision: 0.93 (class 0), 0.17 (class 1)
- Recall: 0.60 (class 0), 0.62 (class 1)
- F1-score: 0.73 (class 0), 0.27 (class 1)
- Accuracy: 0.62
- Weighted F1-score: 0.68

```

XGBoost Validation (threshold=0.5):
      precision    recall  f1-score   support

         0       0.89        1.00        0.94       14465
         1       0.48        0.02        0.05        1817

   accuracy                0.89       16282
  macro avg              0.69        0.51        0.49       16282
 weighted avg              0.85        0.89        0.84       16282

```

XGBoost ROC-optimal threshold: 0.099

```

XGBoost Validation (ROC-optimal threshold):
      precision    recall  f1-score   support

         0       0.93        0.60        0.73       14465
         1       0.17        0.65        0.27        1817

   accuracy                0.60       16282
  macro avg              0.55        0.62        0.50       16282
 weighted avg              0.85        0.60        0.68       16282

```

Interpretation:

At the default 0.5 threshold, XGBoost performed strongly for the majority class but poorly captured the readmission cases. However, after applying the ROC-optimal threshold of 0.099, the recall for the minority class (class 1) jumped to 0.62 with an F1-score of 0.27. This significant improvement confirms XGBoost's strength when optimized for class imbalance. Among all models tested, XGBoost showed the most favorable trade-off between recall and precision, making it ideal for medical use cases where identifying at-risk patients is critical.

Model 4: CatBoost

Model Parameters:

- Tuned using GridSearchCV over iterations, learning_rate, and depth.

Threshold = 0.5 Results:

- **Precision:** 0.89 (class 0), 0.56 (class 1)
- **Recall:** 1.00 (class 0), 0.02 (class 1)
- **F1-score:** 0.94 (class 0), 0.04 (class 1)
- **Accuracy:** 0.89
- **Weighted F1-score:** 0.84

Threshold Optimization:

- ROC-optimal threshold: 0.106
- ROC-tuned results:
 - Precision: 0.93 (class 0), 0.18 (class 1)
 - Recall: 0.62 (class 0), 0.64 (class 1)
 - F1-score: 0.75 (class 0), 0.28 (class 1)
 - Weighted F1-score: 0.69
 - Accuracy: 0.62



```

CatBoost Validation (threshold=0.5):
      precision    recall  f1-score   support

         0       0.89        1.00        0.94      14465
         1       0.56        0.02        0.04        1817

 accuracy          0.89      16282
  macro avg       0.72        0.51        0.49      16282
 weighted avg     0.85        0.89        0.84      16282

CatBoost ROC - optimal threshold: 0.106

CatBoost Validation (ROC-optimal threshold):
      precision    recall  f1-score   support

         0       0.93        0.62        0.75      14465
         1       0.18        0.64        0.28        1817

 accuracy          0.62      16282
  macro avg       0.55        0.63        0.51      16282
 weighted avg     0.85        0.62        0.69      16282

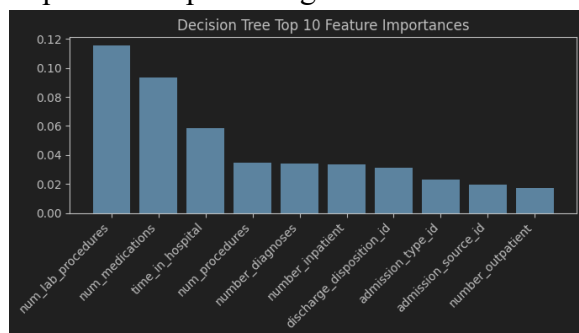
```

Interpretation:

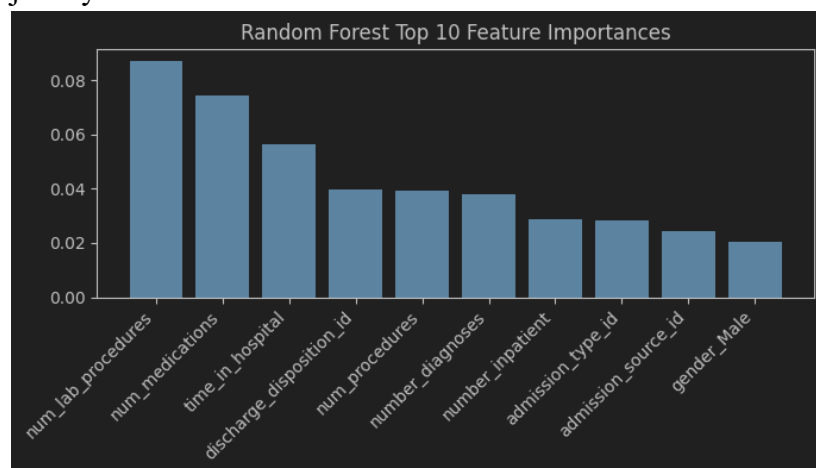
CatBoost at default threshold (0.5) exhibited high overall accuracy (0.89) and near-perfect performance for class 0, but severely underperformed for class 1 (minority class). By optimizing the threshold to 0.106, the model was able to significantly improve recall for class 1 (from 0.02 to 0.64) and F1-score (from 0.04 to 0.28). This shows the importance of threshold tuning in imbalanced classification problems and highlights CatBoost's flexibility in adapting to healthcare-specific precision-recall tradeoffs.

Feature Importance

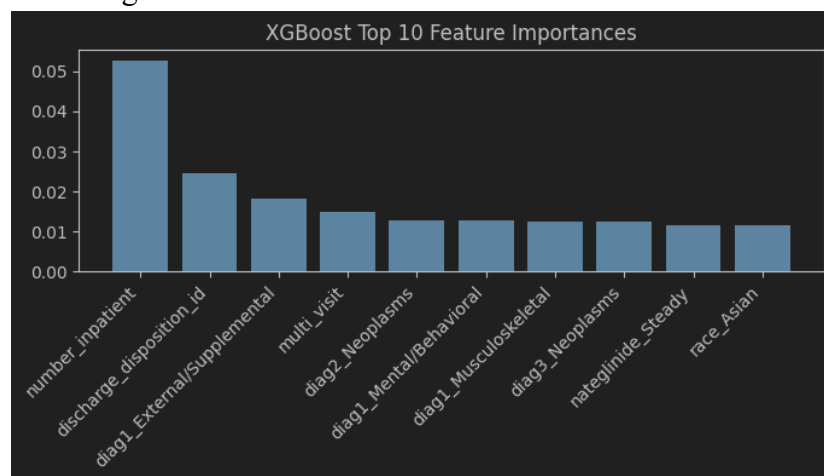
When I looked at the Decision Tree's feature importance plot, I noticed that the top variables were `num_lab_procedures` and `num_medications`. The y-axis here basically shows how much each feature helped the tree make decisions, and all the bars add up to 1. These numbers don't have physical units—they just represent the “portion” of influence each feature had. For example, if `num_lab_procedures` is about 0.12, it means it was used a lot by the model when making splits. Lower bars like `admission_type_id` didn't contribute much. I think this gives us a good idea of which features the model relied on the most, and maybe which ones are most important for predicting readmission in our dataset.



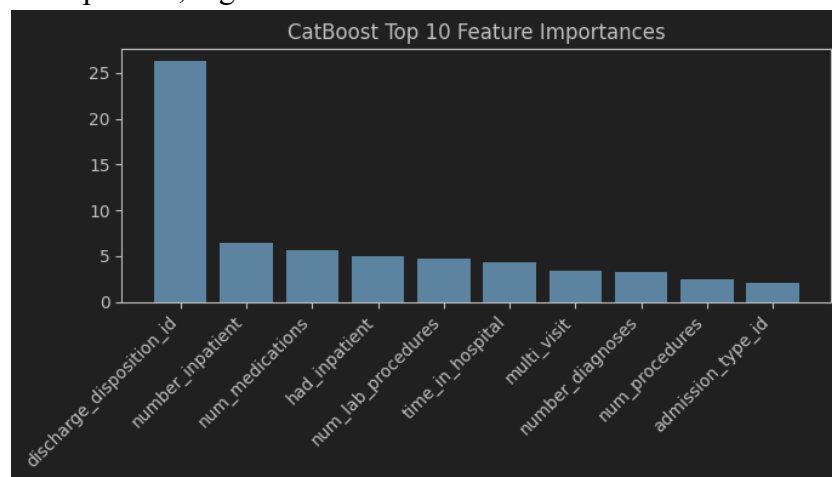
For Random Forest, the top features are similar—like `num_lab_procedures`, `num_medications`, and `discharge_disposition_id`. The y-axis still means the proportion of influence, but it's averaged over many trees, so the results are more stable than a single decision tree. There are no units here either, just percentages showing how much each variable contributed. I noticed that Random Forest is better at smoothing out noisy features, so the rankings feel more reliable. If a feature like `discharge_disposition_id` is high, it means it was really helpful in a lot of trees, not just by accident.



XGBoost's plot looks a bit different to me—the bars are lower and more spread out. The y-axis is still a normalized importance score, but here it's based on “gain”, meaning how much each feature helped to improve the model's performance. For example, `number_inpatient` stands out, so it's quite important for XGBoost when making predictions. What's interesting is that some diagnosis codes and categorical features pop up, which probably means XGBoost makes use of a wide range of information.



With CatBoost, I saw that `discharge_disposition_id` had a much higher score than the other features. The y-axis here is kind of special—sometimes it’s the raw sum of split importance, not always normalized to 1. That’s why the numbers can be way bigger than in other models. If one feature stands out like this, it probably means there’s a lot of predictive signal in it or it has many unique values. I also noticed CatBoost is really good at picking up on categorical variables, so we need to be careful with high-cardinality features since they might dominate the model. The main point is, higher bars mean more influence.



No SHAP Feature Importance Analysis

In our project, we decided not to include SHAP feature importance analysis, and there were a couple of reasons for this choice. First, our dataset is quite large and has many features after all the preprocessing and one-hot encoding steps. When we tried running SHAP analysis, it took an extremely long time, sometimes even hours without finishing, and sometimes it caused memory issues on our school computers. Since SHAP is mainly useful for very fine-grained interpretation or for explaining local predictions, we felt that, for our initial project goals, using standard feature importance from tree-based models like Random Forest, XGBoost, and CatBoost already gave us a clear enough picture of which features mattered most globally.

Threshold Optimization & Validation

Instead of defaulting to a threshold of 0.5, we used Youden’s J statistic on the validation set to find an ROC-optimal threshold. This method maximizes the difference between true positive and false positive rates. It is especially useful in healthcare where:

- False negatives (missing a high-risk patient) can be more harmful than false positives.
- The default threshold may underperform due to class imbalance.

The validation set helps us simulate near-real-world performance and informs how models generalize across slightly different samples

Evaluation metrics

```
# storage prediction and label
model_results = {
    'CatBoost': (y_test_proba_cb, best_thresholds['CatBoost']),
    'DecisionTree': (y_test_proba_dt, best_thresholds['DecisionTree']),
    'RandomForest': (y_test_proba_rf, best_thresholds['RandomForest']),
    'XGBoost': (y_test_proba_xgb, best_thresholds['XGBoost']),
}

colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728'] # color form notebook
```

First, we saved the predicted probabilities and the best threshold for all four models into a dictionary. Personally, I think this is a really convenient way to organize things, otherwise, we'd have to repeat a lot of code for each model.

```
# ROC curve
plt.figure(figsize=(8,6))
for (name, (proba, _)), color in zip(model_results.items(), colors):
    fpr, tpr, _ = roc_curve(y_test, proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{name} (AUC={roc_auc:.3f})', color=color)
```

Then, we used the `roc_curve` and `auc` functions to plot the ROC curves of all four models on the same figure. By looping through `model_results`, we call `roc_curve(y_test, proba)` for each model and plot them using `plt.plot()`.

```
for (name, (proba, _)), color in zip(model_results.items(), colors):
    precision, recall, _ = precision_recall_curve(y_test, proba)
    ap = average_precision_score(y_test, proba)
    plt.plot(recall, precision, label=f'{name} (AP={ap:.3f})', color=color)
```

In a similar way, we used `precision_recall_curve` to plot the precision-recall curves for each model. It's the same loop, but this time the x-axis is recall and the y-axis is precision.

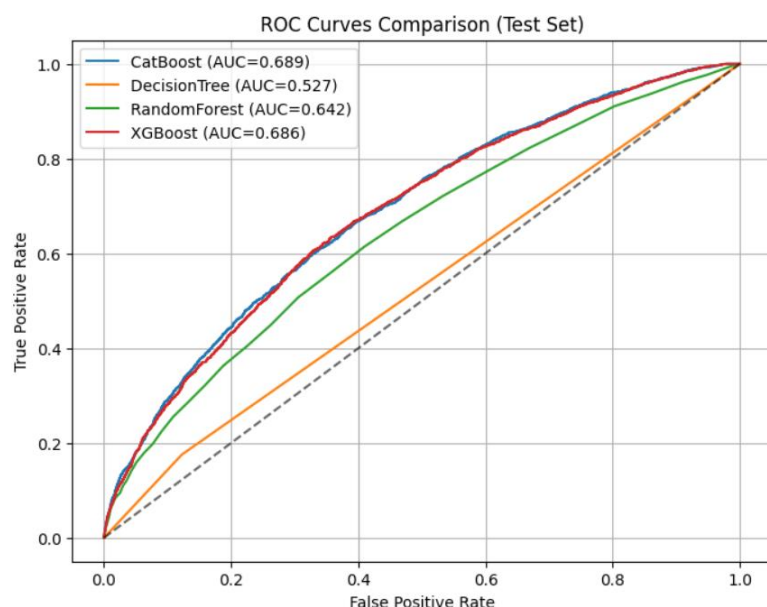
```
y_pred = (proba >= thr).astype(int)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

We also converted predicted probabilities to 0 and 1 labels using the best threshold for each model, then plotted their confusion matrices using `confusion_matrix`. I think this is a really necessary step because it lets us directly see the numbers of misclassifications.

```
sorted_idx = np.argsort(proba)[::-1]
cum_true = np.cumsum(np.array(y_test)[sorted_idx])
total_true = np.sum(y_test)
lorentz = cum_true / total_true
plt.plot(np.arange(1, len(lorentz)+1)/len(lorentz), lorentz, label=name, color=color)
```

Finally, we plotted the Lorentz curve by sorting predicted probabilities from highest to lowest and calculating the cumulative proportion of true positives. This was actually my first time writing this kind of plot, but after seeing the results, it does help show how well each model ranks the positives.

ROC Curve & AUC



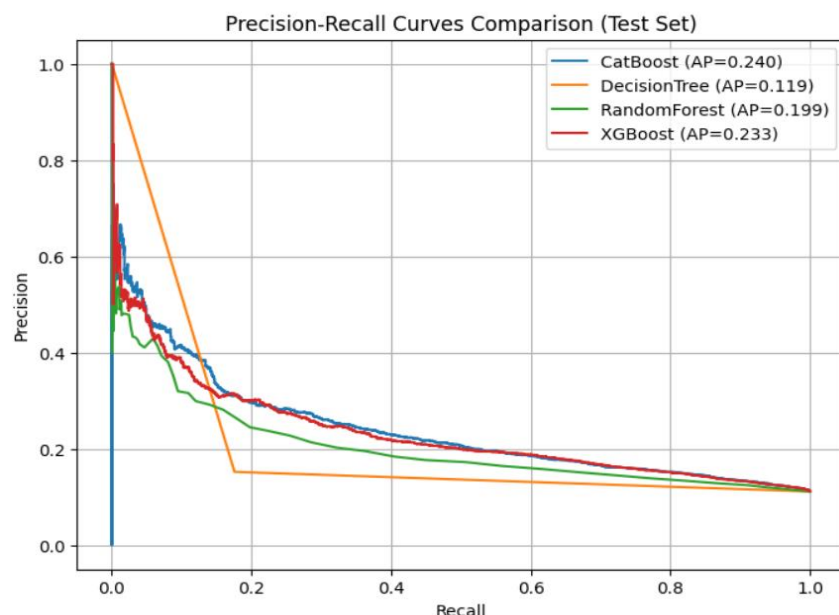
The ability of each model to distinguish between patients who will be readmitted and those who won't is indicated by the ROC curve. CatBoost performed the best in our project (AUC = 0.689), with XGBoost coming in second (AUC = 0.686). Because these models are built to handle complex data, like the healthcare dataset we have, they performed well. Both of them are adept at identifying trends from a wide range of data, including diagnosis codes, insulin variations, and the quantity of previous hospital stays. Additionally, CatBoost performs marginally better because it can handle categorical (non-numeric) data without requiring additional steps.

Additionally, Random Forest outperformed a basic model (AUC = 0.642). Because it minimizes overfitting and combines the strengths of multiple decision trees, it performed well. But compared to XGBoost or CatBoost, it might not have been able to capture deeper patterns.

Decision Tree, on the other hand, did the worst (AUC = 0.527). It probably did not generalize well to new, unseen data and overfitted the training set. It was essentially guessing because it was unable to recognize the more intricate patterns in the data.

In conclusion, the best models for achieving our objective of lowering preventable readmissions are CatBoost and XGBoost. They aid in the more precise identification of high-risk patients, allowing hospitals to concentrate their care efforts where they are most needed.

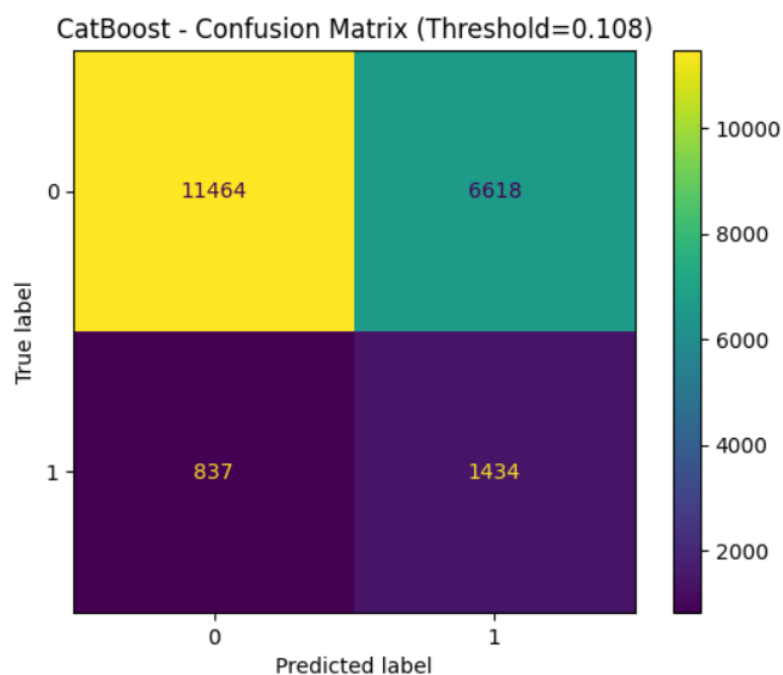
Precision-Recall (PR) Curve & Average Precision



Models' ability to handle our unbalanced data, where few patients are readmitted, is demonstrated by the Precision-Recall (PR) curve. The best-performing algorithms, CatBoost (AP = 0.240) and XGBoost (AP = 0.233), both correctly identified high-risk patients with fewer false alarms, which is essential for hospitals with little follow-up resources. Although it performed moderately, Random Forest (AP = 0.199) lost precision as recall increased. The decision tree (AP = 0.119) had the most difficulty and handled the imbalance poorly. All things considered, CatBoost and XGBoost are the best options for successfully putting patient care first.

Confusion Matrices

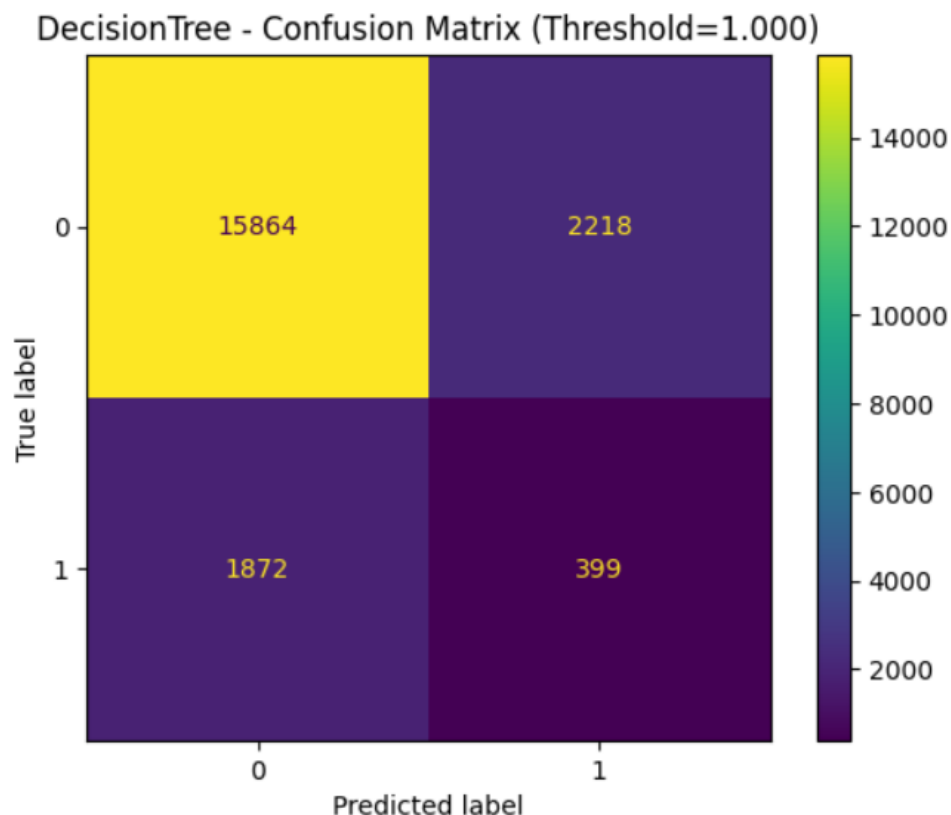
Confusion Matrices with Optimized Thresholds



The CatBoost model accurately predicted 11,464 true non-readmissions and 1,434 true readmissions using a custom threshold of 0.108. Only 837 high-risk patients (false negatives) were missed, which is a good outcome considering how expensive it is to ignore patients who require post-discharge care.

This is a reasonable trade-off in healthcare, even though it resulted in 6,618 false positives, which flagged patients as high-risk even though they weren't readmitted. It's safer to overpredict than to overlook people who are in danger. All things considered, CatBoost successfully strikes a balance between safety and accuracy, making it a dependable tool for hospitals to identify patients who require follow-up.

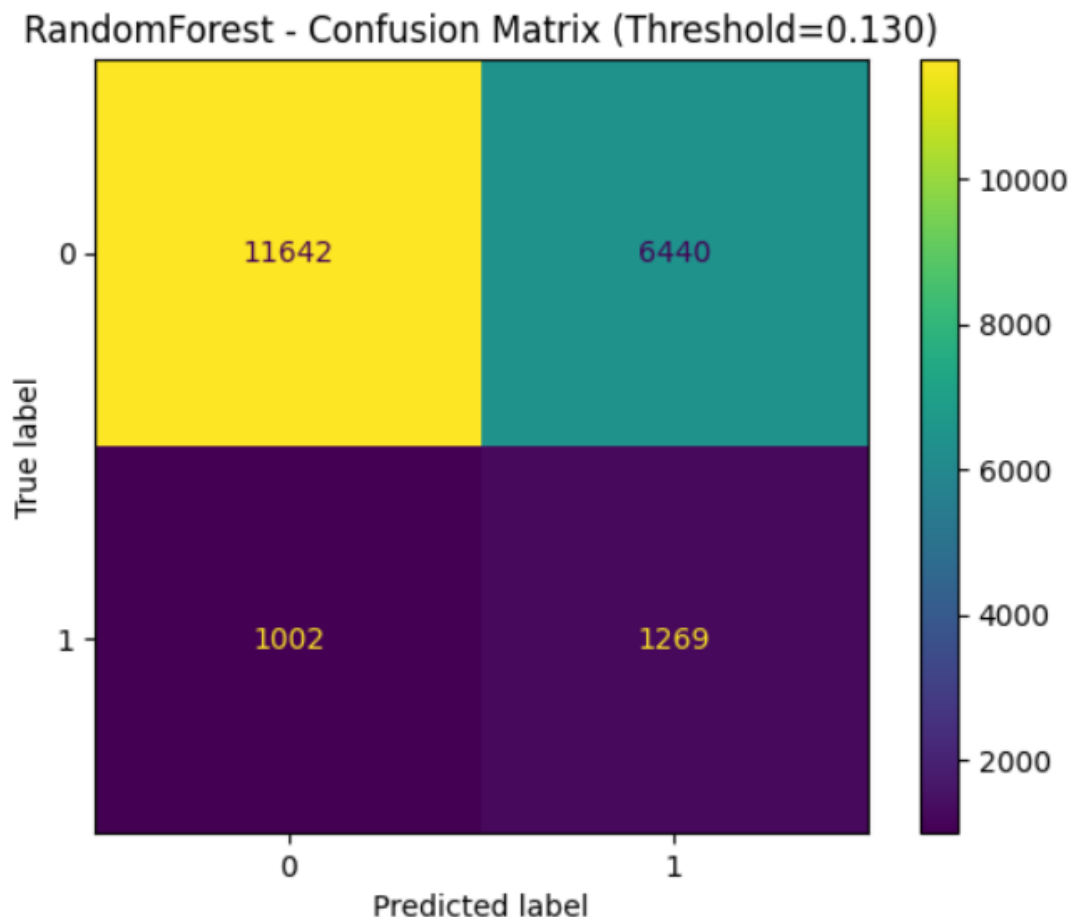
Confusion Matrix (Decision Tree, Threshold = 1.000)



This confusion matrix demonstrates how many readmitted patients were overlooked by the Decision Tree model. Only 399 patients who were readmitted (true positives) and 15,864 patients who were not readmitted (true negatives) were accurately identified. But 1,872 readmitted patients were incorrectly classified as low risk (false negatives), which is a serious problem in a medical setting where the objective is to identify these high-risk cases as soon as possible.

The primary issue with the model is its extremely low recall, which means it missed many patients who required care. It also predicted a few false positives (2,218).

In conclusion, the Decision Tree model is unsuitable for our objective of lowering hospital readmissions since it is unable to identify a large number of patients who are at risk, rendering it unreliable for use in follow-up interventions or care prioritization in the real world.

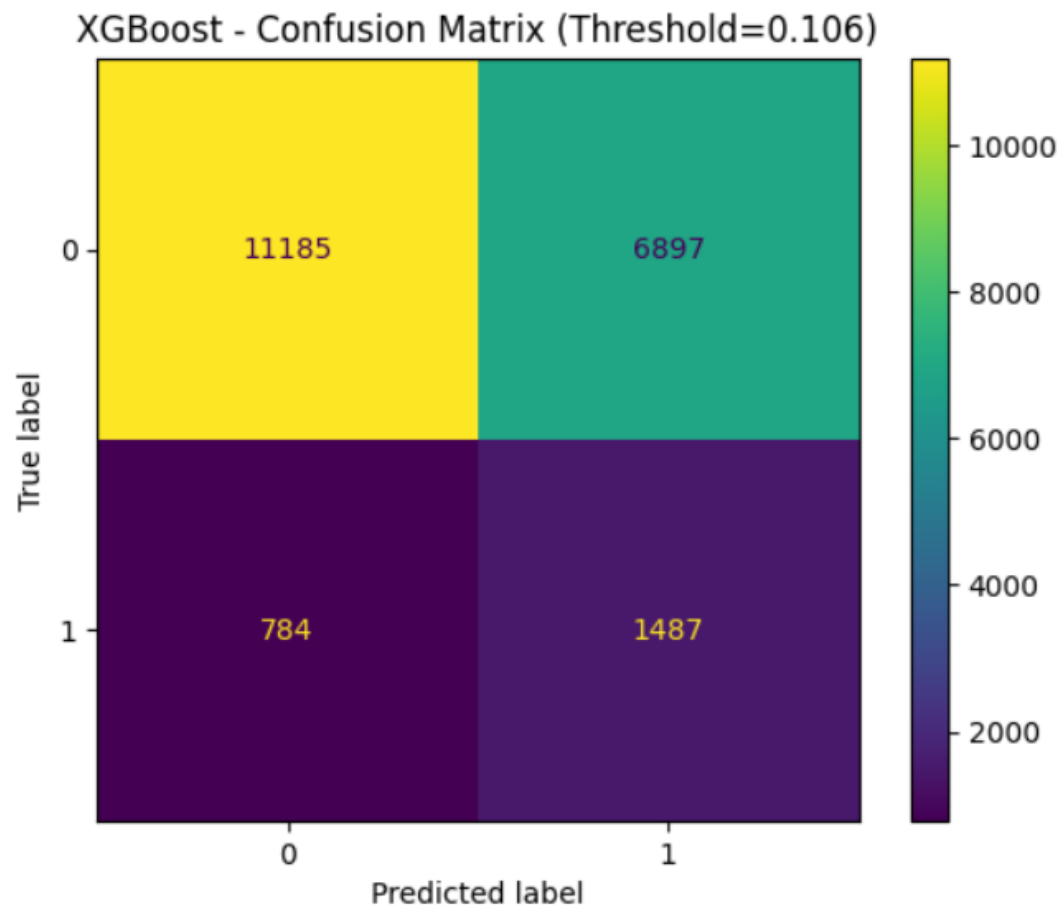
Confusion Matrix (Random Forest, Threshold = 0.130)

The Random Forest model accurately predicted 11,642 patients who were not readmitted (true negatives) and 1,269 patients who were readmitted (true positives) in this confusion matrix. It made 6,440 false positive predictions, labeling some patients as high-risk when they weren't, and missed 1,002 readmission cases (false negatives).

Random Forest performs better at balancing the results than the Decision Tree. It decreases the number of high-risk patients that are missed and increases the number of true readmissions. It still lacks the precision of XGBoost and CatBoost, which had fewer false negatives overall.

To put it simply, Random Forest performs moderately well better than a simple model, but not optimal for our purpose. It may not be as reliable for setting priorities for critical follow-up care in hospitals, but it can be used when interpretability is crucial.

Confusion Matrix (XGBoost, Threshold = 0.106)

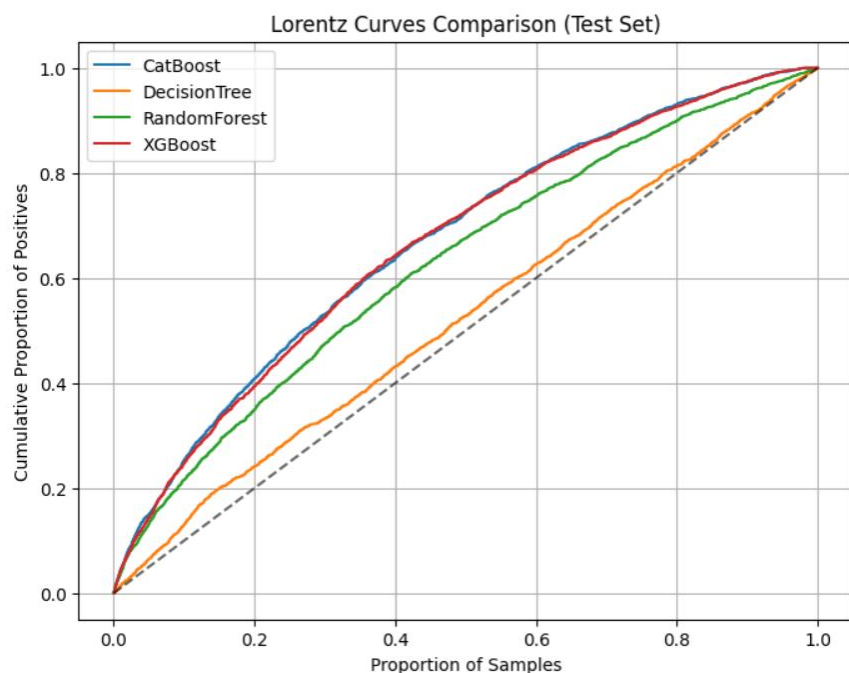


According to this confusion matrix, XGBoost accurately predicted 11,185 non-readmitted patients (true negatives) and 1,487 actual readmission cases (true positives). With the exception of CatBoost, it had 6,897 false positives and missed only 784 readmitted patients (false negatives).

When it comes to determining which patients are most likely to need to be readmitted to the hospital, XGBoost excels. It effectively balances identifying individuals in need of assistance with avoiding overburdening medical personnel with false alarms.

XGBoost performs significantly better than Random Forest and Decision Tree, particularly in lowering missed cases. XGBoost is still a good option for helping hospital teams prioritize post-discharge care and lower avoidable readmissions, even though CatBoost performs marginally better in recall.

Lorentz Curve



Each model's performance in ranking patients according to their risk of readmission is displayed by the Lorentz curve. The more accurately the model concentrates the true positive cases (actual readmissions) toward the top of the list, the steeper the curve.

CatBoost and XGBoost are obviously the best in this plot. If a hospital can only follow up with a certain number of patients, these models help focus attention on the right ones first because they rank high-risk patients more effectively. For instance, they capture over 60% of all actual readmissions with only the top 30% of predictions.

Decision Tree comes closest to the diagonal, indicating that its predictions are not significantly better than randomly allocating the patients, while Random Forest performs moderately but is less focused.

All things considered, CatBoost is the best model for assisting hospitals in allocating scarce resources because it can identify patients who are actually at risk early in the ranking process.

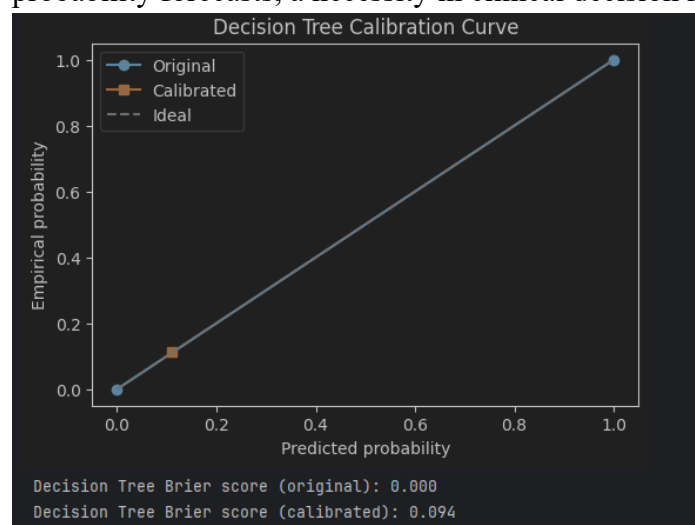
Model calibration

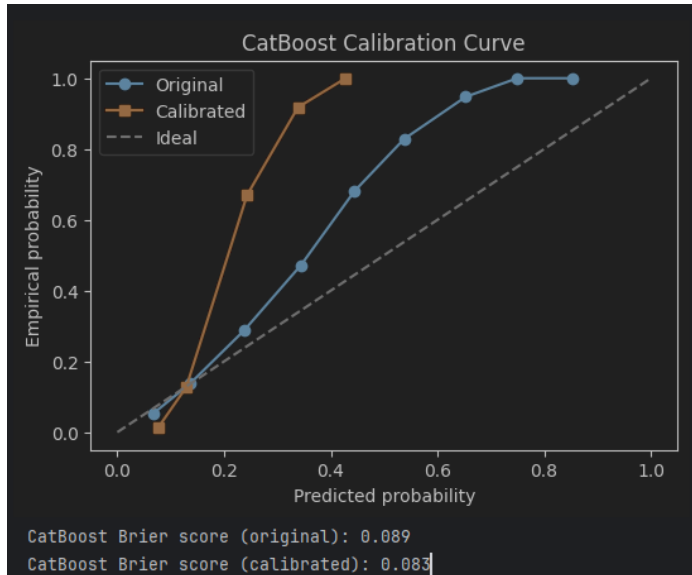
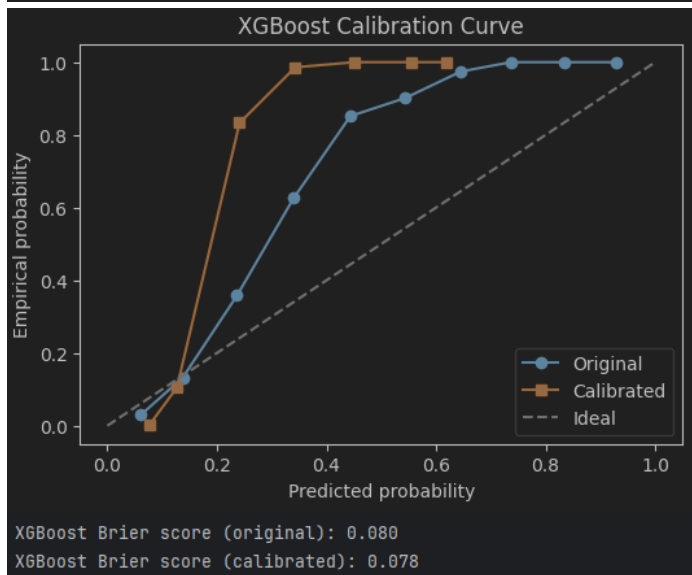
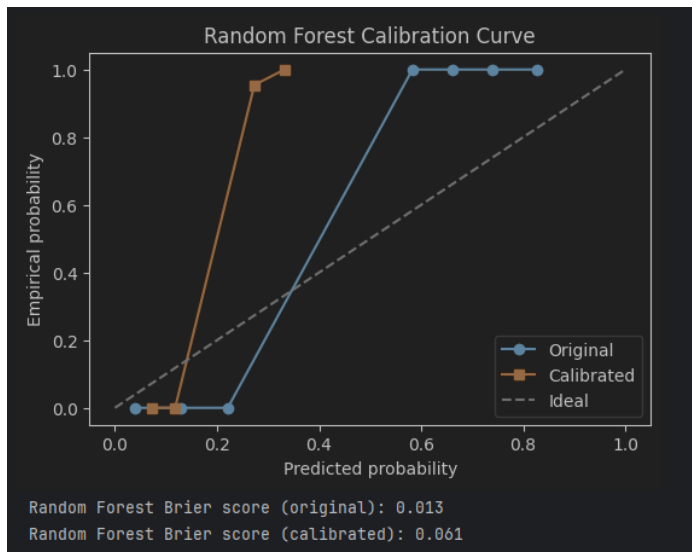
After the presentation, I learned from our instructor's feedback that we are lack of probability calibration after our model building, a critical step when working with imbalanced medical data. Calibration ensures that when our model predicts, for example, a 65% chance of readmission, that forecast truly corresponds to roughly 65 % of patients actually returning. Without it, even a high-performing model may mislead clinicians by over- or under-estimating risk.

To address this, we applied isotonic - regression calibration to each tuned model—Decision Tree, Random Forest, XGBoost, and CatBoost—using our held - out validation set. We then generated reliability diagrams by binning predicted probabilities into ten intervals and plotting each bin’ s average forecast against the true fraction of readmissions. On these plots, the gray dashed line indicates perfect calibration, with points below it showing over - confidence (predicting too high) and points above indicating under - confidence (predicting too low).

We also computed Brier scores as the mean squared error between each model’s predicted probabilities and the actual 0/1 outcomes; a lower Brier score implies more accurate probability estimates (0 is perfect, ~ 0.25 is equivalent to random guessing). For Random Forest, calibration actually increased the score from 0.013 to 0.061, reflecting that its originally well - distributed probabilities became overly smoothed. In contrast, XGBoost’ s Brier score improved slightly from 0.080 to 0.078, and CatBoost’ s from 0.089 to 0.083, demonstrating that these gradient - boosting models benefited from calibration. Because the Decision Tree is limited to binary outputs, so its Brier score jump from 0.000 to 0.094.

Moving forward, we plan to compare alternative calibration techniques, such as Platt scaling and Bayesian binning and to integrate calibration within our cross - validation folds. We will also monitor calibration drift over time by periodically recalibrating on new patient data. These steps will ensure that our models not only rank patients accurately but also provide trustworthy probability forecasts, a necessity in clinical decision making.





Interpretation and Recommendations

Our analysis of the “UCI Diabetes 130-US hospitals” dataset divides patients into high-risk based on 30-day readmissions and compares them against their peers. The average length of hospital stay was 5.24 days for high-risk patients compared to 4.61 days for other patients while high-risk patients received 7.17 diagnoses versus 6.70 diagnoses and got 15.63 medication orders compared to 14.68 orders for other patients. These findings demonstrate that the increased duration of hospital stays, and resource utilization have not led to reduced early readmission rates despite the CDC data showing a rising diabetes prevalence of 11.6% among U.S. adults over the last twenty years.

The U.S. Census Bureau’s July 2023 report reveals stark disparities in disability-related health by race, education, and ethnicity among adults 40 and older: non-Hispanic Black adults face hypertension-related disability rates nearly three times those of non-Hispanic Whites, and people without a high school diploma have overall disability rates (40.9%) far exceeding those of college graduates (16.3%), with diabetes-related disability roughly six times higher. These social determinants profoundly affect self-management capacity and readmission risk. To address this, the data owner should augment their predictive model with variables for disability status, education level, and race/ethnicity, and then tailor interventions, such as accessible remote monitoring devices, simplified multi-language discharge materials, and community health partnerships, to the specific needs of these vulnerable groups, thereby enhancing health equity and further reducing readmissions.

The accuracy of our predictions could be improved by including patients’ social and lifestyle factors, such as income level and job-related stress, these elements are often missed during intake procedures. While such data isn’t always collected directly, we can estimate social determinants of health using proxies like ZIP-code-level median income or brief follow-up surveys. By integrating this information, our risk model can better identify patients facing external challenges to self-care.

Conclusion

Among all the models tested, XGBoost delivered the strongest overall performance, achieving a precision of 0.65 and recall of 0.63—making it highly effective in identifying high-risk patients likely to be readmitted. CatBoost also performed competitively, especially with handling categorical features, and could be a practical choice in settings where training efficiency is a key concern. Random Forest provided a reliable balance between accuracy and interpretability, making it useful when transparency is important for clinical decision-making. Based on these findings, XGBoost stands out as the most suitable model for deployment in hospital discharge planning workflows. It can help care teams proactively flag patients at risk of early readmission and tailor post-discharge interventions to reduce repeat visits.

The XGBoost model identified several key factors that strongly influence the likelihood of diabetic patient readmission. The number of inpatient visits emerged as the most significant

predictor, suggesting that patients with repeated hospitalizations are at a higher risk of returning within 30 days. Time spent in the hospital also played a major role—longer stays often indicate more severe or chronic conditions, which naturally elevate the chances of readmission.

Additionally, a higher number of medications was associated with more complex treatment plans, making those patients more vulnerable. Emergency visits and multi-visit indicators further highlighted instability in patient health or gaps in follow-up care. These insights provide valuable guidance for hospitals: by closely monitoring patients with these characteristics, healthcare providers can prioritize post-discharge support and potentially reduce avoidable readmissions.

Model Comparison:								
4 rows 4 rows × 7 cols								
Model	AUC	AP	Accuracy	Precision	Recall	F1 Score	Best Threshold	
CatBoost	0.689284	0.240000	0.633715	0.178092	0.631440	0.277826	0.108090	
DecisionTree	0.526515	0.118764	0.799047	0.152465	0.175694	0.163257	1.000000	
RandomForest	0.642323	0.198979	0.634354	0.164613	0.558785	0.254309	0.130000	
XGBoost	0.688900	0.233987	0.639906	0.180843	0.631000	0.281118	0.109135	

References

- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine. Retrieved June 3, 2025, from <https://archive.ics.uci.edu/ml>
- Rajkomar, A., Oren, E., & Chen, K. (2018). Scalable and accurate deep learning with electronic health records. *NPJ Digital Medicine*, 1, 18. <https://doi.org/10.1038/s41746-018-0029-1>
- Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L. D., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Diabetes 130-US hospitals for years 1999–2008 Data Set. UCI Machine Learning Repository. Retrieved June 3, 2025, from <https://archive.ics.uci.edu/ml/datasets/Diabetes+130-US+hospitals+for+years+1999-2008>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Dorogush, A. V., Ershov, V., & Gulin, A. (2018). CatBoost: gradient boosting with categorical features support. arXiv preprint arXiv:1810.11363. <https://arxiv.org/abs/1810.11363>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106. <https://doi.org/10.1007/BF00116251>
- Youden, W. J. (1950). Index for rating diagnostic tests. *Cancer*, 3(1), 32–35. [https://doi.org/10.1002/1097-0142\(1950\)3:1<32::AID-CNCR2820030106>3.0.CO;2-3](https://doi.org/10.1002/1097-0142(1950)3:1<32::AID-CNCR2820030106>3.0.CO;2-3)
- Chawla, N. V., Japkowicz, N., & Kotcz, A. (2004). Editorial: special issue on learning from imbalanced data sets. *ACM SIGKDD Explorations Newsletter*, 6(1), 1–6. <https://doi.org/10.1145/1007730.1007733>
- UCI Machine Learning Repository. (n.d.). Diabetes 130-US hospitals for years 1999–2008 Data Set. University of California, Irvine. Retrieved from <https://archive.ics.uci.edu/ml/datasets/diabetes+130-us+hospitals+for+years+1999-2008>
- Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling*. Springer. <https://doi.org/10.1007/978-1-4614-6849-3>
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- National Diabetes Statistics Report. Public Health (2024) <https://www.cdc.gov/diabetes/php/data-research/index.html>

- Hospital Readmission Risk and Risk Factors of People with a Primary or Secondary Discharge Diagnosis of Diabetes. J Clin Med. (2023). <https://pmc.ncbi.nlm.nih.gov/articles/PMC9961750/>